

Manual Técnico: Diseño e Implementación de APIs REST con Node.js

Autor: Departamento de Arquitectura de Software – TechAcademy

Principios de Diseño REST

REST (Representational State Transfer) es un estilo arquitectónico para el diseño de servicios web que fue formalizado por Roy Fielding en su tesis doctoral del año 2000. Sus seis restricciones fundamentales —cliente-servidor, sin estado, caché, interfaz uniforme, sistema en capas y código bajo demanda— proporcionan un marco conceptual robusto para construir APIs escalables e interoperables.

El diseño de una API RESTful exige una reflexión cuidadosa sobre el modelado de recursos, la semántica de los métodos HTTP (GET, POST, PUT, PATCH, DELETE), el uso adecuado de los códigos de estado y la estrategia de versionado.

Implementación con Express.js

Express.js es el framework de referencia para la construcción de servidores HTTP en el ecosistema Node.js. Su arquitectura basada en middleware permite componer pipelines de procesamiento de peticiones de forma declarativa y modular.

La estructura recomendada para un proyecto de mediana complejidad sigue el patrón de capas: rutas (routes), controladores (controllers), servicios (services) y repositorios (repositories). Esta separación de responsabilidades facilita la testabilidad unitaria y la mantenibilidad del código a largo plazo.

Seguridad y Autenticación

La autenticación basada en JSON Web Tokens (JWT) es el mecanismo predominante en APIs REST modernas. El flujo estándar implica la emisión de un access token de corta duración (15-60 minutos) y un refresh token de mayor vigencia (7-30 días), almacenado en una cookie httpOnly para mitigar el riesgo de robo mediante XSS.

La autorización granular se implementa habitualmente mediante RBAC (Role-Based Access Control) o, en escenarios más complejos, ABAC (Attribute-Based Access Control). Es imprescindible aplicar rate limiting, validación exhaustiva de entradas con librerías como Zod o Joi, y auditoría de accesos a recursos sensibles.